

Adaptive Tutoring System

Complete Platform Architecture
& Functionality Reference

Version 0.1.0

Monorepo: NestJS API + React Frontend + FastAPI Worker

PostgreSQL | Redis | MinIO | Docker

1. Project Overview

Name: adaptive-tutoring-system (v0.1.0)

Type: Polyglot monorepo with 3 services + 2 shared packages

Purpose: AI-powered adaptive tutoring platform with knowledge component modeling, LLM-integrated content/question generation, assessment delivery, and mastery tracking using Bayesian Knowledge Tracing (BKT).

2. Tech Stack

Layer	Technologies
Frontend	React 18.3, Vite 5.4, TypeScript 5.5, Tailwind CSS 4.1, Reac
Backend API	NestJS 10.4, TypeScript 5.5, Prisma 6.19, PostgreSQL 16, Red
Worker	FastAPI 0.115, Python 3.12, Uvicorn, psycpg2, Pydantic 2.9
Storage	MinIO (S3-compatible blob storage)
DevOps	Docker Compose, GitHub Actions CI/CD, TurboRepo 2.3, pnpm 10

3. Monorepo Structure

```
capstone/
+-- apps/
|   +-- api/           # NestJS backend (port 3000) - 24 modules, 132+ endpoints
|   +-- web/           # React frontend (port 5173) - 17 routes, 35+ components
|   +-- worker/        # FastAPI worker (port 8000) - async grading + mastery
+-- packages/
|   +-- shared/        # Zod schemas & TypeScript types (35+ schemas)
|   +-- config/        # ESLint + Prettier shared configs
+-- e2e/               # Playwright E2E tests
+-- infra/             # Infrastructure scripts
+-- docker-compose.yml # Full stack definition
```

4. Database Schema (PostgreSQL + Prisma ORM)

4.1 Enums (14 total)

- **UserRole:** student, teacher, admin
- **QuestionType:** text, drawing, mixed, MCQ_SINGLE, MCQ_MULTI, STRUCTURED
- **AttemptStatus:** in_progress, submitted, grading, graded
- **GradedBy:** auto, manual
- **EventStatus:** pending, processing, processed, failed
- **CourseStatus:** DRAFT, PUBLISHED
- **CourseVisibility:** PUBLIC, PRIVATE
- **ModuleItemType:** PAGE, PDF, LINK, ASSESSMENT
- **EnrollmentStatus:** ACTIVE, DROPPED
- **DraftStatus:** DRAFT, APPROVED, REJECTED
- **ChunkingStrategy:** FIXED_SIZE, PARAGRAPH, SEMANTIC
- **GenerationJobStatus:** PENDING, RUNNING, COMPLETED, FAILED
- **EvaluationRunStatus:** PENDING, RUNNING, COMPLETED, FAILED
- **ProposedKCStatus:** PROPOSED, APPROVED, ARCHIVED

4.2 Core Models (30+ tables)

Authentication & Users

- **User:** id, email, passwordHash, name, role, llmProvider, encryptedApiKey, llmModel, createdAt, updatedAt

Course Structure

- **Course:** id, title, description, teacherId, status (DRAFT/PUBLISHED), visibility (PUBLIC/PRIVATE), bannerBlobKey, orderIndex, archivedAt (soft delete)
- **Topic:** id, courseId, title, description, orderIndex, generationJobId
- **CourseModule:** id, courseId, topicId, title, orderIndex, generationJobId
- **ModuleItem:** id, moduleId, type (PAGE/PDF/LINK/ASSESSMENT), title, orderIndex, contentMdx, pdfBlobKey, pdfFilename, pdfSize, url, assessmentId, learningOutcomes (JSON), estimatedMinutes
- **PageContentVersion:** id, itemId, version, contentJson (BlockDocument JSON), source (human/ai/rollback), authorId, jobId

Question Bank & Assessments

- **Question:** id, courseId, topicId, version, prompt, stem (JSON rich blocks), type, maxScore, options (JSON MCQ), correctOptionId, correctAnswer (JSON), explanation (JSON), difficulty (1-5), bloomsLevel, kclIds (UUID[]), pagelIds (UUID[]), tags, status, createdBy, sourceType, provenance
- **Assessment:** id, courseId, title, description, mode (practice/test), isPublished, settings (JSON: timeLimit, maxAttempts, showSolutions, shuffle)
- **AssessmentQuestion:** id, assessmentId, questionId, orderIndex, points, section

Student Attempts & Grading

- **Attempt:** id, studentId, questionId, assessmentId, status, textResponse, selectedOptionIds (JSON), strokesJson (JSON), drawingBlobUrl, workingStrokes (JSON), workingBlobUrl, currentScore, autoFeedback, submittedAt
- **GradingResult:** id, attemptId, score, feedback, gradedBy (auto/manual), graderId

Enrollment

- **Enrollment:** id, studentId, courseId, status (ACTIVE/DROPPED), enrolledAt. Unique: studentId+courseId

Knowledge Components (KC)

- **KnowledgeComponent:** id, topicId, code, label, description (legacy model)
- **ProposedKC:** id, courseId, name, description, status (PROPOSED/APPROVED/ARCHIVED), confidenceLevel (LOW/MEDIUM/HIGH), createdBy (ai/human), generationJobId, relatedToKcId, evidence (JSON), pagels, topicId, subtopicId
- **KcEdge:** id, courseId, fromKcId, toKcId, relationship (prerequisite/related/builds_on), weight, createdBy
- **KcContentMapping:** id, courseId, kcId, pagId, strength (LOW/MEDIUM/HIGH), blockIds, createdBy
- **QuestionKc:** id, questionId, kcId, weight
- **KcEvidence:** id, gradingResultId, kcId, isCorrect, score

Mastery Tracking (BKT-Ready)

- **UserMastery:** id, userId, kcId, probabilityKnown (default 0.5), totalAttempts, correctAttempts, lastAttemptAt, pLearn, pGuess, pSlip, pTransit

RAG & Document Management

- **SourceDocument:** id, courseId, uploadedById, title, filename, blobKey, mimeType, sizeBytes, pageCount, chunkingStrategy, chunkCount, indexedAt
- **DocumentChunk:** id, documentId, chunkIndex, content, pageNumber, tokenCount, embedding (JSON)

LLM Content Generation

- **ContentDraft:** id, courseId, createdById, title, contentMdx, citations (JSON), status (DRAFT/APPROVED/REJECTED), version, parentDraftId
- **LlmGenerationJob:** id, courseId, userId, status, jobType, inputPayload, outputPayload, retrievedChunkIds, errorMessage, model, promptTokens, completionTokens, durationMs
- **LlmAuditLog:** id, draftId, courseId, userId, action, model, promptTokens, completionTokens, durationMs, inputPayload, outputPayload

Evaluation & Quality Assurance

- **EvaluationRun:** id, courseId, userId, status, config (JSON), summary (JSON)
- **PageEvalResult:** id, runId, itemId, itemTitle, scores (JSON: formatting/equations/pedagogy/rigor/overall), issues (JSON array), mathIssues, fixesApplied
- **KcEvaluationRun:** id, courseId, userId, status, scope (JSON), depth (quick/deep), results (JSON)
- **CurriculumCoverageRun:** id, courseId, userId, status, syllabusText, parsedOutcomes, outcomeKcMaps, coverageResults
- **PublishGateRun:** id, courseId, userId, checks (JSON), overallPass, overridden, overrideReason, published
- **KnowledgeVersion:** id, courseId, version, changeSummary, changeType, source, authorId, snapshotKcs, snapshotEdges, snapshotMaps

Visualization & Events

- **KcGraphLayout:** id, courseId (unique), layoutJson (JSON: nodes with x/y, viewBox)
- **EventQueue:** id, topic, payload (JSON), status, processedAt

5. Complete API Endpoints (132+)

5.1 Auth (3 endpoints)

Method	Path	Roles	Description
POST	/auth/register	Public	Register user (throttled 5/min)
POST	/auth/login	Public	Login (throttled 10/min)
GET	/auth/me	JWT	Get current user

5.2 Courses (9 endpoints)

Method	Path	Roles	Description
POST	/courses	teacher	Create course
GET	/courses	Any	List courses (role-filtered)
GET	/courses/catalog	Any	Public course catalog
GET	/courses/:id	Any	Get course details
PATCH	/courses/:id	teacher	Update course
POST	/courses/:id/duplicate	teacher	Deep clone course
POST	/courses/:id/publish	teacher	Publish course
POST	/courses/:id/unpublish	teacher	Unpublish course
DELETE	/courses/:id	teacher	Soft delete course

5.3 Topics (5 endpoints)

Method	Path	Roles	Description
POST	/topics	teacher	Create topic
GET	/topics	Any	List topics (query: courseId)
GET	/topics/:id	Any	Get topic
PATCH	/topics/:id	teacher	Update topic
DELETE	/topics/:id	teacher	Delete topic

5.4 Modules (4 endpoints)

Method	Path	Roles	Description
POST	/courses/:cId/modules	teacher	Create module
PATCH	/courses/:cId/modules/:id	teacher	Update module
DELETE	/courses/:cId/modules/:id	teacher	Delete module
POST	/courses/:cId/modules/reorder	teacher	Reorder modules

5.5 Module Items (9 endpoints)

Method	Path	Roles	Description
POST	/modules/:mId/items	teacher	Create item (PAGE/PDF/LINK/ASSESSMENT)
PATCH	/modules/:mId/items/:id	teacher	Update item
DELETE	/modules/:mId/items/:id	teacher	Delete item
POST	/modules/:mId/items/reorder	teacher	Reorder items
POST	/items/:id/upload-url	teacher	Get S3 presigned upload URL
GET	/items/:id/download-url	Any	Get download URL
GET	/items/:id/versions	teacher	Get version history
GET	/items/:id/versions/:vId	teacher	Get specific version
POST	/items/:id/versions/:vId/rollback	teacher	Rollback to version

5.6 Enrollments (6 endpoints)

Method	Path	Roles	Description
POST	/enrollments	teacher	Enroll student
POST	/enrollments/self	student	Self-enroll in public course
POST	/enrollments/:cId/drop	student	Drop course
GET	/enrollments	teacher	List enrollments
GET	/enrollments/my	student	My enrollments
DELETE	/enrollments/:id	teacher	Remove enrollment

5.7 Questions (10 endpoints)

Method	Path	Roles	Description
POST	/questions	teacher	Create question
GET	/questions	teacher	List (filters: course,topic,type,status,etc)
GET	/questions/course/:cId	Any	Course questions
POST	/questions/bulk-import	teacher	Bulk import
POST	/questions/bulk-status	teacher	Bulk status update
GET	/questions/:id	Any	Get question
PATCH	/questions/:id	teacher	Update question
DELETE	/questions/:id	teacher	Delete question
POST	/questions/:id/duplicate	teacher	Duplicate question
POST	/questions/:id/archive	teacher	Archive question

5.8 AI Question Generation (2 endpoints)

Method	Path	Roles	Description
POST	/questions/generate	teacher	Generate via LLM+RAG
POST	/questions/generate/save	teacher	Save generated questions to bank

5.9 Assessments (10 endpoints)

Method	Path	Roles	Description
POST	/assessments	teacher	Create assessment
GET	/assessments	Any	List assessments
GET	/assessments/available	student	Available assessments
GET	/assessments/:id	Any	Get assessment
PATCH	/assessments/:id	teacher	Update assessment
DELETE	/assessments/:id	teacher	Delete assessment
POST	/assessments/:id/questions	teacher	Add question
DELETE	/assessments/:id/questions/:qid	teacher	Remove question
POST	/assessments/:id/reorder	teacher	Reorder questions
PATCH	/assessments/:id/questions/:qid	teacher	Update question item

5.10 Attempts (8 endpoints)

Method	Path	Roles	Description
POST	/attempts	student	Create attempt
GET	/attempts	teacher	List all attempts
GET	/attempts/my	student	My attempts
GET	/attempts/review	teacher	Pending review
GET	/attempts/:id	Any	Get attempt
PATCH	/attempts/:id	student	Update attempt (auto-save)
POST	/attempts/:id/submit	student	Submit attempt
POST	/attempts/:id/grade	teacher	Manual grade

5.11 Blob Storage (5 endpoints)

Method	Path	Roles	Description
GET	/blobs/presign/upload	Public	Presigned S3 upload URL
GET	/blobs/presign/download	Public	Presigned S3 download URL
PUT	/blobs/:ald/:type	Public	Upload blob
GET	/blobs/:ald/:type	Public	Download blob
DELETE	/blobs/:ald/:type	Public	Delete blob

5.12 RAG & LLM (13 endpoints)

Method	Path	Roles	Description
GET	/courses/:cld/documents	teacher	List documents
POST	/courses/:cld/documents	teacher	Upload document
DELETE	/documents/:dld	teacher	Delete document
POST	/documents/:dld/rechunk	teacher	Rechunk document
POST	/courses/:cld/rag/query	teacher	RAG query
POST	/courses/:cld/rag/generate	teacher	Generate content draft
GET	/courses/:cld/drafts	teacher	List drafts
GET	/drafts/:dld	teacher	Get draft
POST	/drafts/:dld/approve	teacher	Approve draft
POST	/drafts/:dld/reject	teacher	Reject draft
GET	/courses/:cld/llm-logs	teacher	LLM audit logs
GET	/llm-settings	teacher	Get LLM settings
POST	/llm-settings	teacher	Save LLM API key/model

5.13 AI Course Structure (4 endpoints)

Method	Path	Roles	Description
POST	/admin/courses/:cld/generate-structure	teacher	Generate structure (curriculum/level based)
POST	/admin/courses/:cld/apply-structure/:jld	teacher	Apply generated structure
GET	/admin/courses/:cld/structure-jobs	teacher	List jobs
GET	/admin/courses/:cld/structure-jobs/:jld	teacher	Get job

5.14 AI Page Content (3 endpoints)

Method	Path	Roles	Description
POST	/admin/ai/suggest-prompt	teacher	Get prompt suggestion
POST	/admin/pages/:pld/generate-content	teacher	Generate single page
POST	/admin/pages/generate-content-batch	teacher	Batch generate (max 20)

5.15 Content Evaluation (5 endpoints)

Method	Path	Roles	Description
GET	/courses/:cld/evaluation/tree	teacher	Page tree for selector
POST	/courses/:cld/evaluation/run	teacher	Start evaluation
GET	/courses/:cld/evaluation/runs	teacher	List runs
GET	/courses/:cld/evaluation/runs/:rld	teacher	Get run results
POST	/courses/:cld/evaluation/apply-fixes	teacher	Apply auto-fixes

5.16 Mastery & KC Legacy (5 endpoints)

Method	Path	Roles	Description
GET	/me/mastery	student	My mastery by topic
GET	/students/:id/mastery	teacher	Student mastery
POST	/me/revise-worksheet	student	Generate revision worksheet
POST	/kcs	teacher	Create KC
GET	/kcs	teacher	List KCs

5.17 Proposed KCs (10 endpoints)

Method	Path	Roles	Description
GET	/proposed-kcs	teacher	List KCs (courseId, status)
GET	/proposed-kcs/stats	teacher	KC stats
GET	/proposed-kcs/similar-pairs	teacher	Find duplicates
GET	/proposed-kcs/health	teacher	Health indicators
GET	/proposed-kcs/:id	teacher	Get KC
PATCH	/proposed-kcs/:id	teacher	Update KC
POST	/proposed-kcs/:id/approve	teacher	Approve KC
POST	/proposed-kcs/:id/archive	teacher	Archive KC
POST	/proposed-kcs/merge	teacher	Merge KCs
DELETE	/proposed-kcs/:id	teacher	Delete KC

5.18 KC Mappings (7 endpoints)

Method	Path	Roles	Description
GET	/kc-mappings	teacher	List mappings
GET	/kc-mappings/by-kc/:kcId	teacher	By KC
GET	/kc-mappings/by-page/:pId	teacher	By page
GET	/kc-mappings/summary	teacher	Bidirectional summary
POST	/kc-mappings	teacher	Upsert mapping
PATCH	/kc-mappings/:id	teacher	Update strength
POST	/kc-mappings/auto-generate	teacher	Auto-generate from evidence

5.19 KC Graph (12 endpoints)

Method	Path	Roles	Description
POST	/kc-graph/generate	teacher	AI-generate graph
GET	/kc-graph	teacher	Full graph
POST	/kc-graph/edges	teacher	Add edge
PATCH	/kc-graph/edges/:id	teacher	Update edge
DELETE	/kc-graph/edges/:id	teacher	Remove edge
GET	/kc-graph/topological-order	teacher	Topological sort
GET	/kc-graph/cycles	teacher	Detect cycles
POST	/kc-graph/layout	teacher	Save layout
GET	/kc-graph/layout	teacher	Load layout
PATCH	/kc-graph/hierarchy/:kcld	teacher	Update KC hierarchy
POST	/kc-graph/hierarchy/batch	teacher	Batch hierarchy
GET	/kc-graph/hierarchy-options	teacher	Available topics/modules

5.20 KC Evaluation (4 endpoints)

Method	Path	Roles	Description
POST	/courses/:cld/kc-evaluation/run	teacher	Start KC evaluation
GET	/courses/:cld/kc-evaluation/runs/:rld	teacher	Get run
GET	/courses/:cld/kc-evaluation/runs	teacher	List runs
GET	/courses/:cld/kc-evaluation/runs/:rld/export-json	teacher	Export

5.21 Curriculum Coverage (7 endpoints)

Method	Path	Roles	Description
POST	/courses/:cld/curriculum-coverage/run	teacher	Start analysis (syllabusText)
GET	/courses/:cld/curriculum-coverage/runs	teacher	List runs
GET	/courses/:cld/curriculum-coverage/runs/:rld	teacher	Get run
POST	/courses/:cld/curriculum-coverage/runs/:rld/mapping	teacher	Update mapping
DELETE	/courses/:cld/curriculum-coverage/runs/:rld/mapping	teacher	Remove mapping
POST	/courses/:cld/curriculum-coverage/runs/:rld/reanalyze	teacher	Re-analyze
GET	/courses/:cld/curriculum-coverage/runs/:rld/export-js	teacher	Export

5.22 Knowledge Versioning (6 endpoints)

Method	Path	Roles	Description
GET	/courses/:cld/knowledge-versions	teacher	List versions
GET	/courses/:cld/knowledge-versions/:vld	teacher	Get snapshot
GET	/courses/:cld/knowledge-versions/:vld/diff	teacher	Diff vs current
GET	/courses/:cld/knowledge-versions/compare	teacher	Diff two versions
POST	/courses/:cld/knowledge-versions/:vld/restore	teacher	Restore version
GET	/courses/:cld/knowledge-versions/:vld/export	teacher	Export

5.23 Publish Gate (4 endpoints)

Method	Path	Roles	Description
POST	/courses/:cld/publish-gate/check	teacher	Dry run checks
POST	/courses/:cld/publish-gate/publish	teacher	Publish if pass
POST	/courses/:cld/publish-gate/publish-override	teacher	Force publish with reason
GET	/courses/:cld/publish-gate/audit	teacher	Audit history

5.24 WebSocket Gateway

- **join:** Student joins room student:{studentId}
- **grade_completed:** Real-time grading notification to student with score/feedback

6. Frontend Architecture (React + Vite + TypeScript)

6.1 Routes (17 routes)

Public Routes

- /login - Email/password login
- /register - Register with role selection (student/teacher)
- /health - Health check status page

Teacher Routes (teacher, admin)

- /teacher - Dashboard with stat cards
- /teacher/courses - Course listing/management
- /teacher/courses/:courseId - Course builder (tabs: overview, content, sources, evaluate, knowledge, publish, settings)
- /teacher/studio/:courseId - Content studio with block editor
- /teacher/questions - Question bank (filter/create/AI generate)
- /teacher/assessments - Assessment builder
- /teacher/review - Student submission review
- /teacher/attempt/:attemptId - Grade individual attempt
- /teacher/ai-settings - LLM provider/key/model config

Student Routes

- /student - Dashboard (mastery %, weak areas, revision worksheet)
- /student/catalog - Browse/enroll in courses
- /student/courses - My enrolled courses
- /student/courses/:courseId - View course content
- /student/assessments - Available assessments
- /student/attempt/:attemptId - Take assessment (text/drawing/MCQ)
- /student/results - View grades and feedback

6.2 State Management

- **AuthContext:** user state, token, login/register/logout, auto-validate via /auth/me, WebSocket room join for students
- **ToastContext:** success/error/info notifications, 4s auto-dismiss
- **Local state:** Heavy useState/useCallback/useMemo. No Redux/Zustand/Recoil.

6.3 Key Components (35+)

Block Editor

TipTap-based rich content editor with drag-drop block reordering via dnd-kit. Block types: Text, Image, Video, Diagram, Callout, Divider. Auto-save capability.

Drawing Canvas

HTML5 Canvas for drawing questions: pen/eraser tools, adjustable color/width, stroke history, undo/redo, PNG export.

AI Generation Components

- **QuestionGenerationModal:** Config (type/quantity/difficulty/Blooms/KC/source) -> generating -> review
- **GenerateStructureWizard:** Source selection -> config -> preview -> apply
- **PromptComposerModal:** AI page content with scope preference
(module_only/module_then_course/course_only)

Knowledge Component Panels

- **KcStudioPanel:** KC CRUD, approve/archive, merge similar, health metrics, similarity detection
- **KcGraphStudioPanel:** Force-directed graph visualization, colored by subtopic, edge management
- **KcMappingPanel:** KC-Page mappings, auto-generate from evidence, add/edit strength
- **KcEvaluationDashboard:** KC evaluation metrics and coverage analysis
- **CurriculumCoveragePanel:** Syllabus-KC mapping, gap analysis, orphan/overrepresented KC detection
- **KnowledgeVersionPanel:** Version history, snapshots, diffs, restore
- **LearningPathPanel:** Visual learning path with prerequisites
- **PublishGatePanel:** Pre-publish checklist

Other Components

- **MDXRenderer:** MDX compilation with KaTeX math + Mermaid diagrams
- **SourceSelector:** RAG source document selection
- **VersionHistoryPanel:** Page content version comparison
- **BulkProgressPanel:** Progress indicator for bulk operations

6.4 API Client & WebSocket

`apiFetch<T>()` - Core fetch wrapper with JWT injection, 401 auto-redirect, JSON parsing. Convenience: `api.get/post/patch/delete/upload`.

Socket.IO client connects on student login, joins `student:{id}` room, listens for `grade_completed` events for real-time score updates.

7. Worker Service (Python FastAPI)

7.1 Event Processing Flow

Poll PostgreSQL event_queue (every 2s) for 'grade_submission' events:

1. Update attempt status -> 'grading'
2. Grade attempt (string matching against rubric answer_key)
3. Insert grading result into database
4. Update attempt status -> 'graded'
5. Update KC mastery using EMA ($\alpha=0.3$)
6. Publish 'grade_completed' event
7. Mark original event as processed

7.2 Grading Logic

- MCQ: Auto-graded on NestJS backend before reaching worker
- Text/Structured: Case-insensitive string matching against rubric answer_key
- Drawing: Requires manual teacher review
- Missing rubric: Returns 'Manual review required'

7.3 Mastery Update (EMA Algorithm)

$P(\text{known}) = \alpha * \text{current_score} + (1 - \alpha) * \text{old_probability}$
 $\alpha = 0.3$, initial $P = 0.5$, correct threshold = 0.5

For each KC linked to the question:

- Insert KC evidence record (append-only audit trail)
- Update/Create UserMastery record with EMA formula
- Increment totalAttempts and correctAttempts counters

8. Shared Package (@ats/shared)

35+ Zod schemas providing runtime validation and TypeScript types shared across the monorepo:

- **User schemas:** User, CreateUser, Login, AuthResponse
- **Course schemas:** Course, Topic, Question, MCQOption, Enrollment + Create/Update variants
- **Assessment schemas:** Assessment, AssessmentQuestion, Attempt, GradingResult, Stroke/StrokePoint + Create/Update/Submit variants, ManualGrade
- **Mastery schemas:** KnowledgeComponent, CreateKc, QuestionKc, UserMastery, KcEvidence, KcMasteryItem, MasteryByTopic
- **Question Generation:** GenerateQuestionsRequest/Response, GeneratedQuestion/Option/Tags/Provenance, SaveGeneratedQuestionsRequest
- **Event Bus:** GradeSubmissionPayload, GradeCompletedPayload
- **Enums:** UserRole, QuestionType, BloomsLevel, AttemptStatus, EventStatus, SourceMode, Difficulty

9. Security & Infrastructure

- JWT Auth with Passport.js, Bearer token from Authorization header
- Role-based access control via @Roles() decorator + RolesGuard
- Rate limiting via Throttler with Redis store (default 30/60s, auth: 5-10/min)
- ZodValidationPipe for request body validation
- GlobalExceptionHandler: HttpException, ZodError, PrismaClientKnownRequestError
- Cascade deletes on all foreign keys
- Soft deletes for courses (archivedAt), questions (status), KCs (status)
- Per-user encrypted LLM API keys stored in database
- Docker Compose for full stack: api, web, worker, postgres, redis, minio
- GitHub Actions CI/CD: lint, typecheck, integration tests, E2E tests
- TurboRepo for build orchestration with dependency-aware caching

10. Key Architectural Patterns

- **Event-driven async processing:** PostgreSQL event queue with polling worker for decoupled grading
- **Knowledge graph modeling:** Proposed KCs with edges (prerequisite/related/builds_on), topological sort, cycle detection
- **Version control:** Page content versions, knowledge graph snapshots with diff/restore
- **RAG pipeline:** Document upload -> chunking (fixed/paragraph/semantic) -> embedding -> retrieval -> LLM generation
- **BKT-ready mastery:** EMA-based probability updates with BKT parameters (pLearn, pGuess, pSlip, pTransit)
- **Block-based content:** Structured JSON blocks (text, image, video, diagram, callout, divider)
- **Multi-step AI workflows:** Course structure -> page content -> evaluation -> publish gate
- **Audit trail:** LLM audit logs tracking all AI calls with token usage and duration
- **Monorepo code sharing:** Shared Zod schemas + TypeScript types consumed by all apps via workspace packages